Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр
Тема: «Ссылочный тип. Инициализация классов, конструкторы и деструкторы»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Изучить использование ссылочного типа в пользовательских классах; приобрести практические навыки программирования конструкторов (по умолчанию, с параметрами, копирования) и деструкторов; освоить представление структуры класса в виде UML-диаграммы.

Ход работы:

Задание: На основе класса Student (предметная область «Студент»), реализованного в предыдущих работах, добавить в него: динамический массив типа `int[]` (вариант для чётного номера в списке группы), выделяемый в конструкторах с помощью `new[]` и освобождаемый в деструкторе с помощью `delete[]`; конструктор копирования и оператор присваивания `=`, исключающие поверхностное копирование и утечки памяти; ссылочное поле; продемонстрировать передачу и возврат по ссылке и по значению; реализовать три способа возврата массива из метода (указатель, ссылка на массив, ссылка на элемент); построить UML-диаграмму класса.

Описание разработанного класса:

Относительно предыдущей работы статический массив оценок заменён динамическим, добавлены ссылочное поле, деструктор и оператор присваивания. Поля `course` и `group`, метод `TO_SHOW` и методы доступа сохранены. Размер массива задан константой `SIZE = 5`. В функции `main()` работа класса демонстрируется на объектах `s1`, `s2`, `s3` (раздел 1) и `a`, `b`, `d` (разделы 2-4).

1. Поля класса.

- `int course`, `int group` - курс и группа;
- `int* grades` - указатель на динамический массив оценок из `SIZE` элементов; память выделяется в конструкторах, освобождается в деструкторе;
- `int& rating` - ссылочное поле: ссылка на внешнюю переменную (рейтинг группы). Ссылка инициализируется только в списке

инициализации конструктора и позже не может быть перепривязана к другому объекту;

- `static int alive` - счётчик существующих объектов: увеличивается в конструкторах, уменьшается в деструкторе. После уничтожения всех объектов он равен 0, что подтверждает отсутствие утечек;
- `static int defaultRating` - переменная, к которой привязывается ссылка в конструкторе по умолчанию (ссылка не может быть «пустой»).

2. Конструкторы и деструктор.

- `Student()` - по умолчанию: курс 1, группа 0, массив из нулей, `rating` связан с `defaultRating`;
- `Student(int c, int g, const int arr[], int& r)` - с параметрами: выделяет массив и копирует в него значения из `arr`; `rating` связывается с переменной `r`, переданной по ссылке;
- `Student(const Student& other)` - копирования: выделяет новый массив и копирует элементы (глубокое копирование). Ссылка `rating` связывается с тем же объектом, что и у `other`, так как ссылка является псевдонимом и копируется как привязка к той же переменной;
- `~Student()` - деструктор: освобождает массив с помощью `delete[]`;
- `Student& operator=(const Student& other)` - присваивание: проверка самоприсваивания, освобождение старого массива, выделение нового и копирование элементов. Ссылка `rating` не перепривязывается (в C++ это невозможно, а запись `rating = other.rating` изменила бы значение внешней переменной), поэтому копируются только `course`, `group` и оценки.

Деструктор, конструктор копирования и оператор присваивания определены вместе (правило трёх): при определении любого из них

компилятор-генерируемые версии для остальных привели бы к поверхностному копированию указателя и двойному освобождению памяти.

3. Методы.

- `TO_SHOW()` - вывод объекта, `TO_GET_Course()`, `TO_GET_Group()`, `TO_GET_Sum()`, `TO_GET_Rating()` - методы получения значений;
- `TO_SET_Rating(int r)` - записывает значение через ссылку, то есть изменяет внешнюю переменную;
- `int* TO_GET_Array()` - возврат указателя на массив;
- `int (&TO_GET_ArrayRef())[SIZE]` - возврат ссылки на массив; так как массив динамический, ссылка получается приведением `reinterpret_cast<int(&)[SIZE]>(*grades)`;
- `int& TO_GET_Element(int i)` - возврат ссылки на отдельный элемент; при неверном индексе возвращается ссылка на вспомогательную статическую переменную;
- `static int TO_GET_Alive()` - число существующих объектов.

4. Демонстрация работы.

4.1. Динамический массив. Создаются `s1` (по умолчанию), `s2` (с параметрами) и `s3` (копия `s2`). Изменение элемента `s3` не затрагивает `s2`: массивы у объектов независимы (глубокое копирование). После `s1 = s2` изменение `s1` также не влияет на `s2`; самоприсваивание `s1 = s1` не нарушает данные. После выхода из блока вызываются три деструктора, а счётчик `alive` равен 0.

4.2. Ссылочная переменная. Объекты `a` и `b` связаны с переменной `groupRating`. Изменение `groupRating` из `main()` сразу видно через оба объекта, а вызов `a.TO_SET_Rating(90)` изменяет саму переменную и виден через `b`. После `a = d` рейтинг `a` остаётся связанным с `groupRating`, а `otherRating` (переменная объекта `d`) не изменяется.

4.3. Передача и возврат. При передаче по значению (`changeByValue(Student s)`) вызывается конструктор копирования, функция меняет копию, которая уничтожается при выходе, поэтому оригинал не

меняется. При передаче по ссылке (`changeByRef(Student& s)`) копия не создаётся, и изменения видны в оригинале. Функция `bestByRef` возвращает ссылку на один из переданных объектов, поэтому запись через результат меняет оригинал; `bestByValue` возвращает копию (вызывается конструктор копирования), изменения копии оригинал не затрагивают.

4.4. Способы возврата массива. Указатель (`int*`) позволяет читать и изменять элементы, но не хранит размер: `sizeof(p)` равен размеру указателя (8 байт на 64-разрядной платформе). Ссылка на массив сохраняет тип с размером: `sizeof(r)` равен 20 байт (5 элементов), по ней работает цикл `for (int x : r)`. Ссылка на элемент (`int&`) позволяет записывать значения напрямую, в том числе в выражениях вида `TO_GET_Element(0)++`.

Код программы:

```
#include <iostream>
#include <cstdlib>
#include <clocale>
using namespace std;

class Student
{
public:
    static const int SIZE = 5;

private:
    int course;
    int group;

    int *grades;

    int &rating;

    static int alive;

    static int defaultRating;

public:
    Student() : rating(defaultRating)
    {
        course = 1;
        group = 0;
        grades = new int[SIZE];
        for (int i = 0; i < SIZE; i++)
        {
```

```
        grades[i] = 0;
    }
    alive++;
    cout << "  [Конструктор по умолчанию] объектов: "
         << alive << endl;
}

Student(int c, int g, const int arr[], int &r) : rating(r)
{
    course = c;
    group = g;
    grades = new int[SIZE];
    for (int i = 0; i < SIZE; i++)
    {
        grades[i] = arr[i];
    }
    alive++;
    cout << "  [Конструктор с параметрами] объектов: "
         << alive << endl;
}

Student(const Student &other) : rating(other.rating)
{
    course = other.course;
    group = other.group;
    grades = new int[SIZE];
    for (int i = 0; i < SIZE; i++)
    {
        grades[i] = other.grades[i];
    }
    alive++;
    cout << "  [Конструктор копирования] объектов: "
         << alive << endl;
}

~Student()
{
    delete[] grades;
    alive--;
    cout << "  [Деструктор] объектов: " << alive << endl;
}

Student &operator=(const Student &other)
{

    if (this != &other)
    {
        delete[] grades;
        course = other.course;
        group = other.group;
        grades = new int[SIZE];
        for (int i = 0; i < SIZE; i++)
        {
```

```
            grades[i] = other.grades[i];
        }
    }
    cout << "  [Оператор =]" << endl;
    return *this;
}

void TO_SHOW() const
{
    cout << "Курс: " << course;
    cout << " | Группа: " << group;
    cout << " | Оценки: ";
    for (int i = 0; i < SIZE; i++)
    {
        cout << grades[i] << " ";
    }
    cout << "| Рейтинг: " << rating << endl;
}

int TO_GET_Course() const { return course; }
int TO_GET_Group() const { return group; }

int TO_GET_Sum() const
{
    int sum = 0;
    for (int i = 0; i < SIZE; i++)
    {
        sum += grades[i];
    }
    return sum;
}

int TO_GET_Rating() const { return rating; }

void TO_SET_Rating(int r) { rating = r; }

int *TO_GET_Array() { return grades; }

int (&TO_GET_ArrayRef()) [SIZE]
{
    return reinterpret_cast<int (&)[SIZE]>(*grades);
}

int &TO_GET_Element(int i)
{

    static int stub = 0;
    if (i < 0 || i >= SIZE)
    {
        stub = 0;
        return stub;
    }
    return grades[i];
```

```
    }

    static int TO_GET_Alive() { return alive; }
};

int Student::alive = 0;
int Student::defaultRating = 0;

void changeByValue(Student s)
{
    s.TO_GET_Element(0) = 1;
    cout << "  внутри функции (копия): ";
    s.TO_SHOW();
}

void changeByRef(Student &s)
{
    s.TO_GET_Element(0) = 1;
    cout << "  внутри функции (оригинал): ";
    s.TO_SHOW();
}

Student &bestByRef(Student &a, Student &b)
{
    return (a.TO_GET_Sum() >= b.TO_GET_Sum()) ? a : b;
}

Student bestByValue(Student &a, Student &b)
{
    return (a.TO_GET_Sum() >= b.TO_GET_Sum()) ? a : b;
}

int main()
{
    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    int g[Student::SIZE] = {8, 9, 7, 10, 9};
    int groupRating = 70;

    cout << "=== 1. Динамический массив ===" << endl;
    {

        Student s1;
        Student s2(2, 29, g, groupRating);
        Student s3(s2);
        cout << "s1: ";
        s1.TO_SHOW();
        cout << "s2: ";
        s2.TO_SHOW();
        cout << "s3: ";
        s3.TO_SHOW();
```

```
        cout << endl
             << "Изменяем оценку №1 у s3 (копия):" << endl;
        s3.TO_GET_Element(0) = 4;
        cout << "s2: ";
        s2.TO_SHOW();
        cout << "s3: ";
        s3.TO_SHOW();

        cout << endl
             << "Присваивание s1 = s2:" << endl;
        s1 = s2;
        cout << "s1: ";
        s1.TO_SHOW();
        s1.TO_GET_Element(4) = 5;
        cout << "s1 после изменения: ";
        s1.TO_SHOW();
        cout << "s2 не изменился:    ";
        s2.TO_SHOW();

        cout << endl
             << "Самоприсваивание s1 = s1:" << endl;
        s1 = s1;
        cout << "s1: ";
        s1.TO_SHOW();
        cout << endl
             << "Выход из блока (освобождение памяти):"
             << endl;
    }
    cout << "Живых объектов: " << Student::TO_GET_Alive() <<
endl;

    cout << endl
         << "=== 2. Ссылочная переменная ===" << endl;
    Student a(2, 29, g, groupRating);
    Student b(a);
    cout << "groupRating = " << groupRating << endl;
    cout << "a: ";
    a.TO_SHOW();

    groupRating = 85;
    cout << "groupRating = 85" << endl;
    cout << "a: ";
    a.TO_SHOW();
    cout << "b: ";
    b.TO_SHOW();

    a.TO_SET_Rating(90);
    cout << "a.TO_SET_Rating(90), groupRating = "
         << groupRating << endl;
    cout << "b: ";
    b.TO_SHOW();

    int otherRating = 50;
```

```
int c[Student::SIZE] = {5, 5, 5, 5, 5};
Student d(1, 15, c, otherRating);
cout << endl
     << "Присваивание a = d (ссылка не перепривязывается):"
     << endl;
a = d;
cout << "a: ";
a.TO_SHOW();
cout << "groupRating = " << groupRating
     << ", otherRating = " << otherRating << endl;

cout << endl
     << "=== 3. Передача и возврат ===" << endl;
cout << "Передача по значению:" << endl;
changeByValue(b);
cout << "  после вызова: ";
b.TO_SHOW();
cout << "Передача по ссылке:" << endl;
changeByRef(b);
cout << "  после вызова: ";
b.TO_SHOW();

cout << endl
     << "Возврат по ссылке:" << endl;
Student &best = bestByRef(a, b);
best.TO_GET_Element(1) = 10;
cout << "a: ";
a.TO_SHOW();
cout << "b: ";
b.TO_SHOW();
cout << "Возврат по значению:" << endl;
Student copy = bestByValue(a, b);
copy.TO_GET_Element(1) = 0;
cout << "copy: ";
copy.TO_SHOW();
cout << "b:    ";
b.TO_SHOW();

cout << endl
     << "=== 4. Способы возврата массива ==="
     << endl;

int *p = b.TO_GET_Array();
p[2] = 6;
cout << "1) указатель, p[2] = 6: ";
b.TO_SHOW();
cout << "   sizeof(p) = " << sizeof(p) << endl;

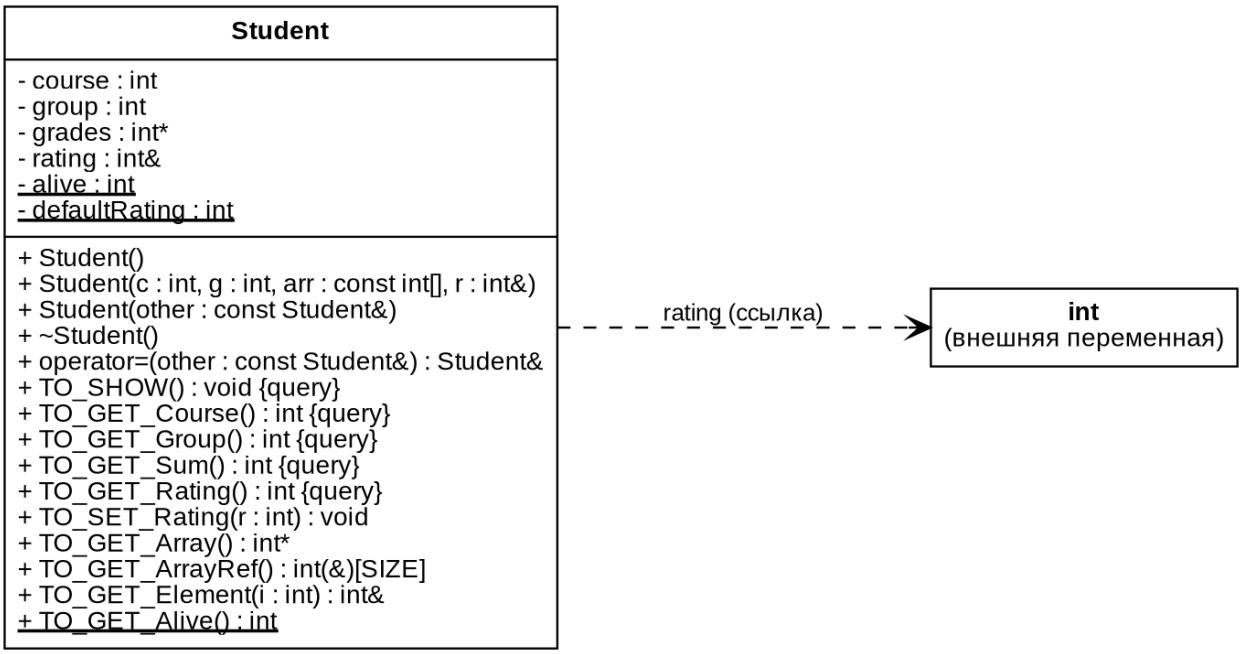
int (&r)[Student::SIZE] = b.TO_GET_ArrayRef();
cout << "2) ссылка на массив: ";
for (int x : r)
    cout << x << " ";
cout << endl;
```

```
    r[3] = 7;
    cout << "   после r[3] = 7: ";
    b.TO_SHOW();
    cout << "   sizeof(r) = " << sizeof(r)
         << " (" << Student::SIZE << " элементов)" << endl;

    int &e = b.TO_GET_Element(4);
    e = 9;
    cout << "3) ссылка на элемент, e = 9: ";
    b.TO_SHOW();
    b.TO_GET_Element(0)++;
    cout << "   TO_GET_Element(0)++: ";
    b.TO_SHOW();

    cout << endl
         << "Выход из main (освобождение памяти):" << endl;
    return 0;
}
```

UML-диаграмма класса:



На диаграмме - обозначает закрытые (`private`), + - открытые (`public`) члены класса; статические члены подчёркнуты. Пунктирная стрелка показывает, что поле `rating` является ссылкой на внешнюю переменную типа `int`, существующую независимо от объекта.

Результат:

1. Динамический массив (выделение, копирование, освобождение):

```
=== 1. Динамический массив ===
  [Конструктор по умолчанию] объектов: 1
  [Конструктор с параметрами] объектов: 2
  [Конструктор копирования] объектов: 3
s1: Курс: 1 | Группа: 0 | Оценки: 0 0 0 0 0 | Рейтинг: 0
s2: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70
s3: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70

Изменяем оценку №1 у s3 (копия):
s2: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70
s3: Курс: 2 | Группа: 29 | Оценки: 4 9 7 10 9 | Рейтинг: 70

Присваивание s1 = s2:
  [Оператор =]
s1: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 0
s1 после изменения: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5 | Рейтинг: 0
s2 не изменился:    Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70

Самоприсваивание s1 = s1:
  [Оператор =]
s1: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5 | Рейтинг: 0

Выход из блока (освобождение памяти):
  [Деструктор] объектов: 2
  [Деструктор] объектов: 1
  [Деструктор] объектов: 0
Живых объектов: 0
```

2. Ссылочная переменная:

```
=== 2. Ссылочная переменная ===
  [Конструктор с параметрами] объектов: 1
  [Конструктор копирования] объектов: 2
groupRating = 70
a: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70
groupRating = 85
a: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 85
b: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 85
a.TO_SET_Rating(90), groupRating = 90
b: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 90
  [Конструктор с параметрами] объектов: 3

Присваивание a = d (ссылка не перепривязывается):
  [Оператор =]
a: Курс: 1 | Группа: 15 | Оценки: 5 5 5 5 5 | Рейтинг: 90
groupRating = 90, otherRating = 50
```

3. Передача и возврат по ссылке и по значению:

```
=== 3. Передача и возврат ===
Передача по значению:
  [Конструктор копирования] объектов: 4
  внутри функции (копия): Курс: 2 | Группа: 29 | Оценки: 1 9 7 10 9 | Рейтинг: 90
  [Деструктор] объектов: 3
  после вызова: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 90
Передача по ссылке:
  внутри функции (оригинал): Курс: 2 | Группа: 29 | Оценки: 1 9 7 10 9 | Рейтинг: 90
  после вызова: Курс: 2 | Группа: 29 | Оценки: 1 9 7 10 9 | Рейтинг: 90

Возврат по ссылке:
a: Курс: 1 | Группа: 15 | Оценки: 5 5 5 5 5 | Рейтинг: 90
b: Курс: 2 | Группа: 29 | Оценки: 1 10 7 10 9 | Рейтинг: 90
Возврат по значению:
  [Конструктор копирования] объектов: 4
copy: Курс: 2 | Группа: 29 | Оценки: 1 0 7 10 9 | Рейтинг: 90
b:    Курс: 2 | Группа: 29 | Оценки: 1 10 7 10 9 | Рейтинг: 90
```

4. Способы возврата массива и освобождение объектов при выходе из `main()`:

```
=== 4. Способы возврата массива ===
1) указатель, p[2] = 6: Курс: 2 | Группа: 29 | Оценки: 1 10 6 10 9 | Рейтинг: 90
   sizeof(p) = 8
2) ссылка на массив: 1 10 6 10 9
   после r[3] = 7: Курс: 2 | Группа: 29 | Оценки: 1 10 6 7 9 | Рейтинг: 90
   sizeof(r) = 20 (5 элементов)
3) ссылка на элемент, e = 9: Курс: 2 | Группа: 29 | Оценки: 1 10 6 7 9 | Рейтинг: 90
   TO_GET_Element(0)++: Курс: 2 | Группа: 29 | Оценки: 2 10 6 7 9 | Рейтинг: 90

Выход из main (освобождение памяти):
  [Деструктор] объектов: 3
  [Деструктор] объектов: 2
  [Деструктор] объектов: 1
  [Деструктор] объектов: 0
```

Вывод: Изучили использование ссылочного типа в пользовательских классах и реализовали класс Student с динамическим массивом, ссылочным полем, конструкторами (по умолчанию, с параметрами, копирования), деструктором и оператором присваивания. Показали, что корректное копирование (правило трёх) исключает поверхностное копирование и утечки памяти: счётчик объектов после уничтожения равен 0. Выяснили, что ссылочное поле должно инициализироваться в списке инициализации конструктора и не может быть переприв язано, а при передаче и возврате по значению создаётся копия объекта, тогда как по ссылке работа идёт с оригиналом. Реализовали три способа возврата массива (указатель, ссылка на массив, ссылка на элемент) и построили UML-диаграмму класса.